

ملخص مختصر لاهم

اساسيات Node.js



اعداد: Youssef Salah

محتوي الملخص

• مقدمة عن Node.js

• إعداد البيئة وتثبيت Node.js

• فهم مفهوم ال Modules في Node.js

• التعامل مع الأحداث (Event Handling) في Node.js

• التعامل مع ال File System (FS) في Node.js

• التعامل مع ال HTTP Server في Node.js

• التعامل مع قواعد البيانات (Databases) في Node.js

• الأداء وتحسين الكفاءة (Performance Optimization) في Node.js

• التعامل مع ال APIs في Node.js

• التعامل مع الأحداث (Event-Driven Architecture)

• ال Middleware في Express

• ال Error Handling في Node.js

• التعامل مع ال Streams في Node.js

• ال Security Basics في Node.js

• التعامل مع ال WebSockets في Node.js

• التعامل مع ال Sessions و JSON Web Tokens (JWT)

• أهم أسئلة ما بعد الملخص ؟

• الخاتمة

ملخص مختصر ل أهم
أساسيات React.js



اعداد: Youssef Salah

قريبا...

مقدمة عن Node.js



مقدمة عن Node.js:

Node.js عبارة عن بيئة تشغيل مفتوحة المصدر مبنية على JavaScript V8 الخاص بجوجل وده معناه إنك تقدر تشغل كود JavaScript برا المتصفح واتعملت عشان تسهل تطوير تطبيقات الويب خصوصًا اللي بتحتاج أداء عالي وتعتمد على ال I/O (زي السيرفرات)

مميزاتها الأساسية:

1. خفيف وسريع:

• بيعتمد على ال Event-driven, non-blocking I/O. وده بيخليه سريع جدًا في التعامل مع العمليات.

2. Single-threaded:

• بيشتغل على خيط واحد (Single-thread). لكن بيعرف يتعامل مع آلاف الطلبات في نفس الوقت

باستخدام ال Event Loop.

3. اللغة الموحدة (JavaScript):

• تقدر تكتب الكود لل Back-end و Front-end بلغة واحدة، وهي JavaScript.

4. إدارة الحزم (npm):

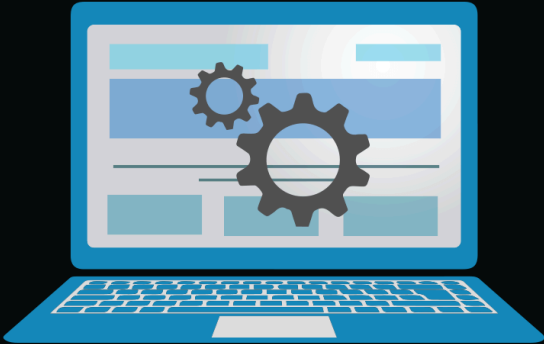
• بييجي مع مدير الحزم الشهير npm اللي بيساعدك تضيف مكتبات وأدوات بسهولة للمشروع.

Youssef Salah



إيه اللي أقدر أعمله بـ Node.js؟

- تطبيقات ويب (زي السيرفرات).
- تطبيقات Real-time زي الشات والألعاب الأونلاين.
- API Services لتقديم بيانات لـ Front-end أو Mobile Apps.
- برامج لسطح المكتب باستخدام أدوات زي Electron.



Web Application Development



Youssef Salah

ليه أتعلم Node.js؟

- مناسب جدًا لو عايز تدخل مجال ال Full Stack Development.
- سهل تتعلمه لو عارف JavaScript.
- أدائه عالي ومناسب للتطبيقات الكبيرة.



Youssef Salah

إعداد البيئة وتثبيت Node.js



إعداد البيئة وتثبيت Node.js

أول حاجة محتاج تعملها هي تثبيت Node.js على جهازك هتروح لموقع Node.js الرسمي: <https://nodejs.org>.
LT (Long Term Support): النسخة المستقرة وبفضل تستخدمها للمشاريع.
Current: النسخة الأحدث، لكنها ممكن تكون غير مستقرة.

بعد التثبيت تأكد إن Node.js و npm (مدير الحزم) اتثبتوا صح
• افتح Terminal أو Command Prompt واكتب:

```
node -v  
npm -v
```

هتظهرلك إصداراتهم، كده التثبيت نجح.

هنيجي بقي لتثبيت محرر اكواد مناسب ؟

أفضل محرر كود تستخدمه مع Node.js هو **VS Code**.

• نزل VS Code من هنا: <https://code.visualstudio.com>.

• ده محرر خفيف وسريع وفيه إضافات كتير مفيدة لكتابة كود Node.js.



Youssef Salah

إنشاء مشروع Node.js جديد

افتح ال Terminal أو Command Prompt وخليك في المكان اللي عايز تعمل فيه المشروع.

- اكتب الأمر ده عشان تنشئ ملف إعدادات المشروع

```
npm init
```

هيسألك شوية أسئلة زي اسم المشروع والوصف والإصدار، وغيره.
لو عايز تتخطى الأسئلة وتسبب الإعدادات الافتراضية، استخدم

```
npm init -y
```

هيتكون ملف اسمه **package.json**، وده ملف مهم جدًا بيحتوي على تفاصيل المشروع والمكتبات اللي هتستخدمها.

وعشان تبدا تشغيل المشروع الاول ليك

- اعمل ملف جديد اسمه app.js في المشروع.
- جرب الكود ده عشان تطبع رسالة Hello Node.js

```
console.log("Hello, Node.js!");
```

شغل الكود بكتابة الأمر ده في ال Terminal

```
node app.js
```

فهم مفهوم الـ

Modules في

Node.js



يعني إيه Modules؟

ال Modules هي طريقة لتنظيم الكود بدل ما تكتب كل الكود في ملف واحد بتقسم الكود لملفات (Modules) وكل ملف بيكون ليه وظيفة معينة.

أنواع ال Modules في Node.js

Core Modules (الأساسية)

- دي مكتبات جاهزة في Node.js زي **http**, **fs**, **path**.
- بتستخدمها مباشرة بدون ما تثبت حاجة.

Local Modules (محلية)

- دي ملفات بتكتبها بنفسك عشان تنظم الكود بتاعك.
- Third-Party Modules (طرف ثالث):
- مكتبات بتثبتها باستخدام npm زي **mongoose**, **express**, **axios**.

استخدام Core Modules مثال لاستخدام **fs** (File System) لقراءة ملف اسمه `example.txt`:

```
const fs = require('fs');

fs.readFile('example.txt', 'utf8', (err, data) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log(data);
});
```

إنشاء واستخدام Local Modules

مثال لو عندك ملف اسمه **math.js** يحتوي على العمليات الحسابية:

```
// math.js
function add(a, b) {
  return a + b;
}

module.exports = { add };
```

وفي ملفك الرئيسي **app.js**:

```
const math = require('./math');

console.log(math.add(5, 3)); // 8
```

Youssef Salah

تثبيت واستخدام Third-Party Modules

مثال على تثبيت مكتبة **axios**:

```
npm install axios
```

استخدام المكتبة في الكود:

```
const axios = require('axios');

axios.get('https://jsonplaceholder.typicode.com/posts')
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.error(error);
  });
```

الكود ده بيحيب بيانات من API وهمي.

Youssef Salah

التعامل مع الأحداث (Event Handling) في Node.js



يعني إيه أحداث (Events)؟

في Node.js، الأحداث هي طريقة للتعامل مع الأكواد التي تحصل بشكل غير متزامن (Asynchronous). الفكرة إنها بتشغل لما يحصل حاجة معينة زي حدث المستخدم (User Action) واستجابة من السيرفر.

2. EventEmitter Class

- مكتبة events في Node.js بتوفر كلاس اسمه EventEmitter.
- الكلاس ده بيسمح لك تصنع أحداث وتتعامل معاها.

3. استخدام EventEmitter

كود بسيط يوضح التعامل مع الأحداث:

```
const EventEmitter = require('events');

const myEmitter = new EventEmitter();

// تعريف الحدث
myEmitter.on('greet', () => {
  console.log('Hello, World!');
});

// إطلاق الحدث
myEmitter.emit('greet');
```

لما تستخدم emit، الحدث اللي اسمه greet بيشتغل وينفذ الكود اللي داخله.

Youssef Salah

تمرير بيانات مع الأحداث

ممكن تبعت بيانات للحدث زي كده:

```
myEmitter.on('sayHello', (name) => {  
  console.log(`Hello, ${name}!`);  
});  
  
myEmitter.emit('sayHello', 'Ahmed');
```

هنا الحدث بياخد اسم المستخدم وبيطبع رسالة ترحيب.

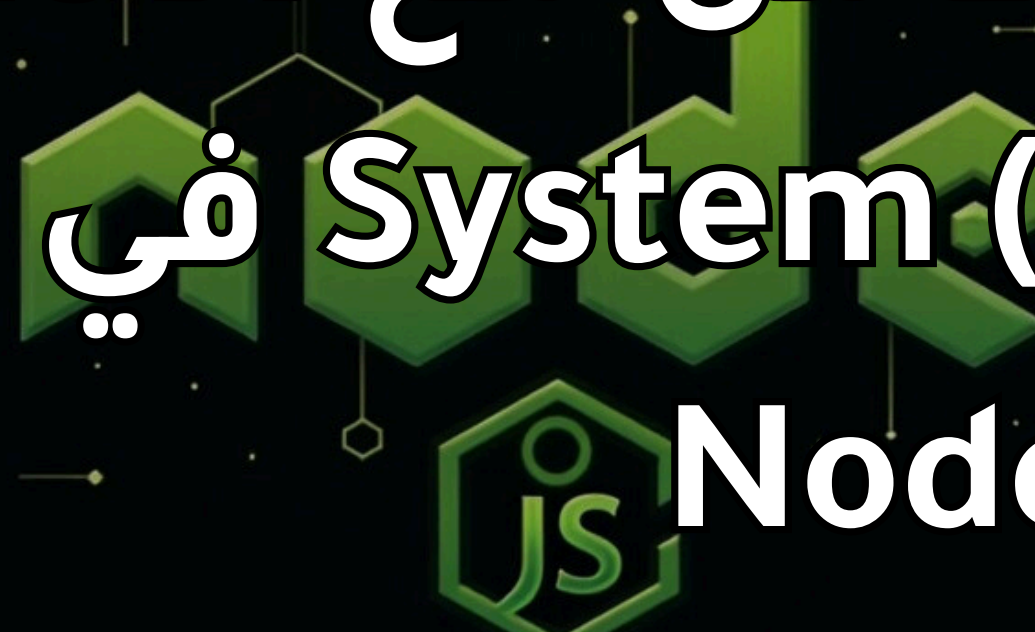
استخدام الأحداث في السيناريوهات العملية

مثال على التعامل مع الملفات باستخدام الأحداث:

```
const fs = require('fs');  
  
const fileEmitter = new EventEmitter();  
  
fileEmitter.on('readFile', (fileName) => {  
  fs.readFile(fileName, 'utf8', (err, data) => {  
    if (err) {  
      console.error('Error reading file:', err);  
    } else {  
      console.log('File content:', data);  
    }  
  });  
});  
  
fileEmitter.emit('readFile', 'example.txt');
```

هنا الحدث بيقراً محتوى ملف معين لما يتم إطلاقه.

التعامل مع الـ File System (FS) في Node.js



يعني إيه File System؟

ال File System هو واجهة بتسمح لك تتعامل مع الملفات والمجلدات (قراءة، كتابة، تعديل، حذف) مباشرة من خلال كود Node.js. مكتبة fs في Node.js هي المسؤولة عن العمليات دي.

أنواع العمليات في File System

- عمليات متزامنة (Synchronous):
 - الكود بيتنفذ خطوة بخطوة، مستني العملية تخلص.
 - عمليات غير متزامنة (Asynchronous):
 - الكود مش بيعتني العملية تخلص وبيكمل تنفيذ باقي الأكواد.
- أمثلة على التعامل مع File System لقراءة ملف:

```
const fs = require('fs');

// قراءة غير متزامنة
fs.readFile('example.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Error reading file:', err);
  } else {
    console.log('File content:', data);
  }
});
```

Youssef Salah

انواع الFile System؟

مثال لكتابة ملف:

```
fs.writeFile('example.txt', 'Hello, Node.js!', (err) => {  
  if (err) {  
    console.error('Error writing file:', err);  
  } else {  
    console.log('File written successfully!');  
  }  
});
```

مثال لحذف ملف:

```
fs.unlink('example.txt', (err) => {  
  if (err) {  
    console.error('Error deleting file:', err);  
  } else {  
    console.log('File deleted successfully!');  
  }  
});
```

Youssef Salah

التعامل مع المجلدات (Directories)

إنشاء مجلد:

```
fs.mkdir('newFolder', (err) => {  
  if (err) {  
    console.error('Error creating folder:', err);  
  } else {  
    console.log('Folder created successfully!');  
  }  
});
```

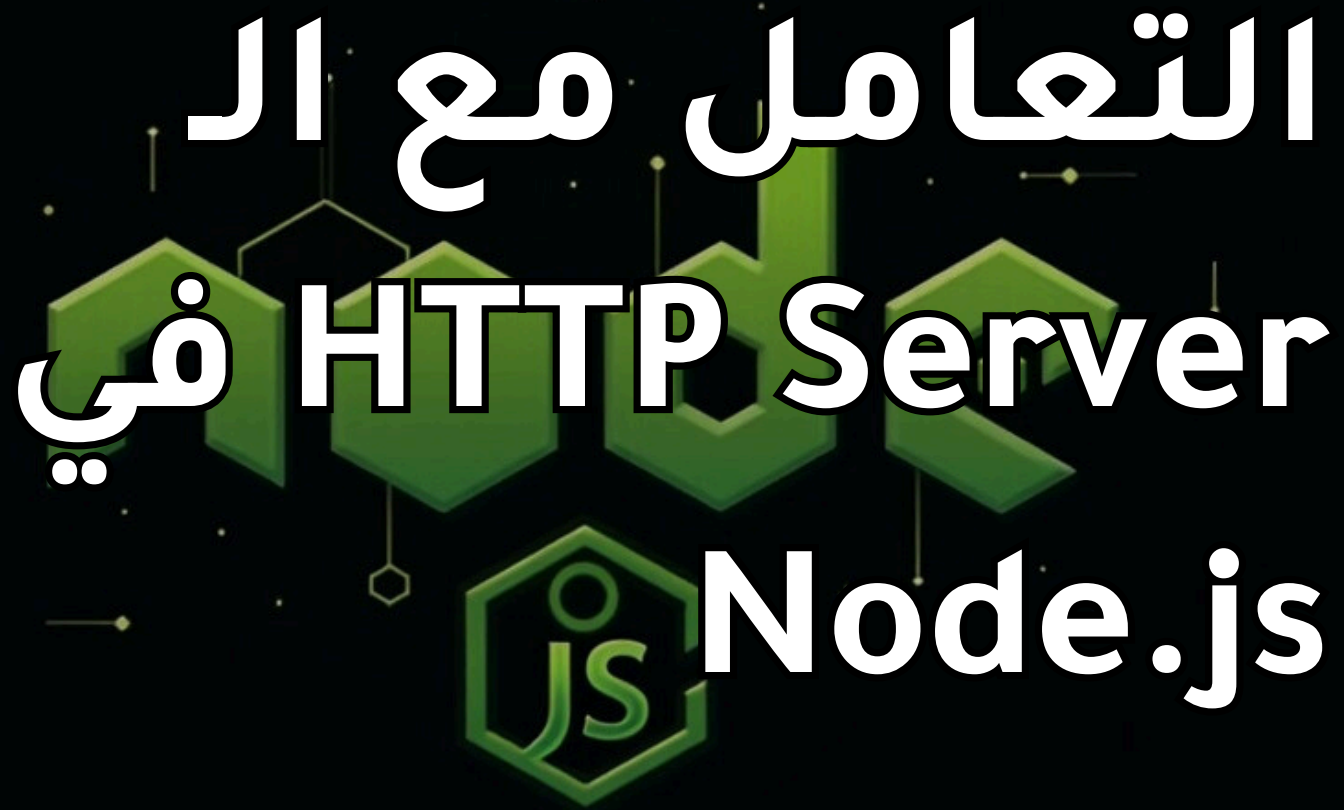
قراءة محتويات مجلد:

```
fs.readdir('.', (err, files) => {  
  if (err) {  
    console.error('Error reading directory:', err);  
  } else {  
    console.log('Directory content:', files);  
  }  
});
```

قراءة محتويات مجلد:

```
fs.rmdir('newFolder', (err) => {  
  if (err) {  
    console.error('Error deleting folder:', err);  
  } else {  
    console.log('Folder deleted successfully!');  
  }  
});
```

التعامل مع الـ HTTP Server في Node.js

The background features a dark blue field with several green hexagons of varying sizes and orientations. Some hexagons are solid green, while others are outlined. A prominent Node.js logo, consisting of a green hexagon with a white 'o' and 'js' inside, is positioned centrally behind the text. Faint white lines and dots are scattered across the background, suggesting a network or digital theme.

يعني إيه HTTP Server؟

ال HTTP Server هو اللي بيخلي التطبيق بتاعك يقدر يستقبل طلبات (requests) من المستخدمين ويرد عليهم (responses) Node.js بتوفر مكتبة اسمها **http** عشان تقدر تبني **Server** بسهولة.

خطوات إنشاء HTTP Server بسيط:

1. استيراد مكتبة http.

2. إنشاء السيرفر باستخدام `createServer`.

3. تحديد البورت اللي السيرفر هيشغل عليه.

مثال عملي ل إنشاء HTTP Server

```
const http = require('http');

// إنشاء السيرفر
const server = http.createServer((req, res) => {
  // تحديد نوع الرد
  res.writeHead(200, { 'Content-Type': 'text/plain' });

  // كتابة الرد
  res.end('Hello, Node.js HTTP Server!');
});

// تشغيل السيرفر على بورت 3000
server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

التعامل مع طلبات مختلفة (Routes)

مثال عملي

```
const server = http.createServer((req, res) => {
  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Welcome to the Home Page!');
  } else if (req.url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('About Us Page');
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('Page Not Found');
  }
});
server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

ملاحظات مهمة:

يمكن تحديد أنواع مختلفة من الـ Content زي JSON بدل النصوص العادية:

```
res.writeHead(200, { 'Content-Type': 'application/json' });
res.end(JSON.stringify({ message: 'Hello, JSON!' }));
```

لو عاوز تبني سيرفر معقد أكثر، ممكن تستخدم مكتبات زي **Express.js** عشان تسهل الشغل.

Youssef Salah

التعامل مع قواعد

البيانات

(Databases) في

Node.js

ليه نحتاج قواعد بيانات؟

عشان تخزن بيانات التطبيق زي بيانات المستخدمين والمنتجات أو أي حاجة تانية محتاجها التطبيق لان Node.js بتدعم التعامل مع أنواع مختلفة زي قواعد البيانات العلائقية (Relational Databases) زي MySQL و PostgreSQL. أو غير العلائقية (NoSQL) زي MongoDB.

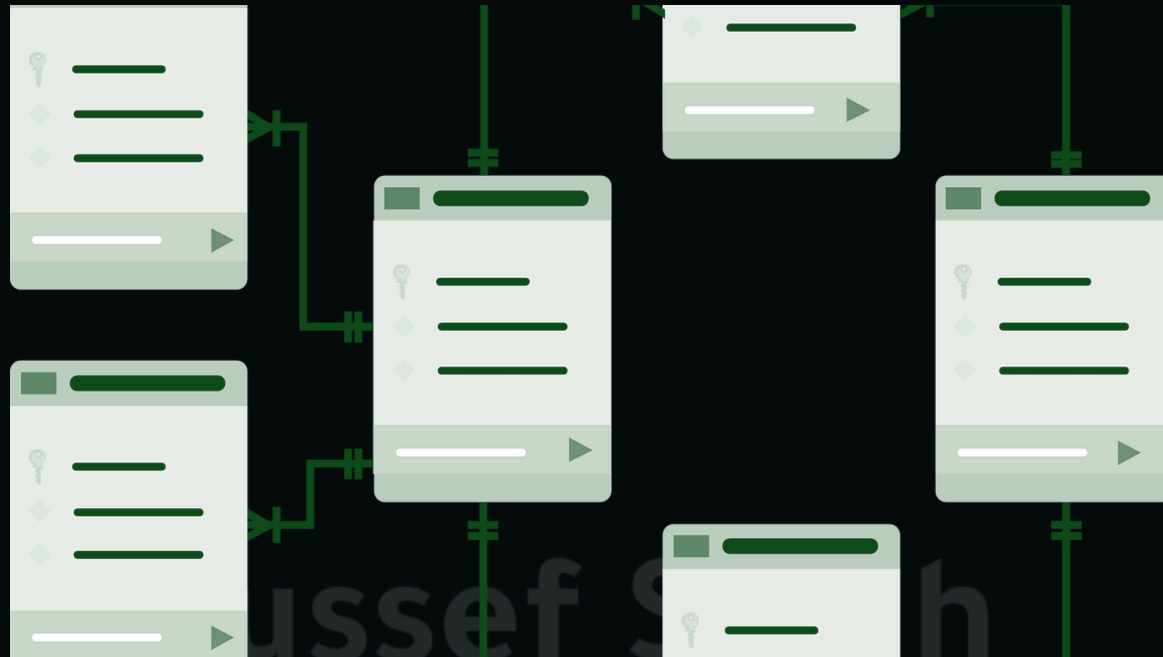
أشهر المكتبات للتعامل مع قواعد البيانات:

:Relational Databases

- مكتبة mysql2 أو pg للتعامل مع MySQL أو PostgreSQL.

:NoSQL Databases

- مكتبة mongoose للتعامل مع MongoDB.



التعامل مع MongoDB باستخدام Mongoose

مثال عملي لتثبيت المكتبة

```
npm install mongoose
```

الاتصال بقاعدة البيانات:

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/mydatabase', {
  useNewUrlParser: true,
  useUnifiedTopology: true,
}).then(() => {
  console.log('Connected to MongoDB!');
}).catch((error) => {
  console.error('Failed to connect to MongoDB:', error);
});
```

تعريف Schema وإضافة بيانات:

```
const userSchema = new mongoose.Schema({
  name: String,
  age: Number,
});

const User = mongoose.model('User', userSchema);

const newUser = new User({ name: 'Ali', age: 25 });
newUser.save().then(() => {
  console.log('User added!');
});
```

التعامل مع MySQL باستخدام mysql2

تثبيت المكتبة:

```
npm install mysql2
```

الاتصال بقاعدة البيانات:

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'testdb',
});

connection.connect((err) => {
  if (err) {
    console.error('Error connecting to MySQL:', err);
    return;
  }
  console.log('Connected to MySQL!');
});
```

تعريف Schema وإضافة بيانات:

```
connection.query('SELECT * FROM users', (err, results) => {
  if (err) throw err;
  console.log(results);
});
```

الأداء وتحسين الكفاءة

Performance)

Optimization) في

Node.js



ليه تحسين الأداء مهم؟

عشان التطبيقات تكون أسرع وتوفر موارد السيرفر وتتعامل مع عدد كبير من المستخدمين في نفس الوقت. Node.js بيستخدم Non-blocking I/O و Event Loop لكن لو الكود مش مكتوب بشكل كويس الأداء بيتأثر.

أفضل ممارسات لتحسين الأداء:

أ. استخدام ال Asynchronous Programming:

- دائماً استخدم **async/await** أو ال **Callbacks** بدل العمليات المتزامنة.

مثال:

```
const fs = require('fs').promises;

async function readFile() {
  try {
    const data = await fs.readFile('file.txt', 'utf8');
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}

readFile();
```

Youssef Salah

استخدام الـ Caching

خزن البيانات التي سيتم طلبها كثير في الذاكرة بدل ما تجيبها كل مرة من قاعدة البيانات.
مثال باستخدام مكتبة node-cache:

```
npm install node-cache
```

```
const NodeCache = require('node-cache');  
const cache = new NodeCache();  
  
const key = 'userData';  
const value = { name: 'Ali', age: 25 };  
  
cache.set(key, value, 100); // ثنية 100  
console.log(cache.get(key));
```

اما لو عاوز تقلل حجم الاستجابات:

لو بتبعت بيانات كثير من الـ API، حاول تبعت بس اللي المستخدم محتاجه.

مثال:

```
app.get('/users', (req, res) => {  
  const users = [  
    { id: 1, name: 'Ali', age: 25 },  
    { id: 2, name: 'Sara', age: 30 },  
  ];  
  res.json(users.map(user => ({ id: user.id, name: user.name })));  
});
```

تحسين الأداء في قواعد البيانات

- استخدم الفهارس (Indexes) في قواعد البيانات.
- قلل الاستعلامات المعقدة (Complex Queries).
- استخدم مكتبات زي **sequelize** أو **mongoose** لتسريع كتابة الكود.

أدوات لمراقبة الأداء:

pm2: لإدارة التطبيقات وتحليل أدائها.

```
npm install pm2 -g  
pm2 start app.js  
pm2 monit
```

clinic: لتحليل أداء التطبيق.

```
npm install pm2 -g  
pm2 start app.js  
pm2 monit
```

نصائح مهمة ؟



- دائماً اختار الطريقة الغير متزامنة لتنفيذ العمليات.
- استخدم الكاش لتقليل الأحمال على قواعد البيانات.
- قلل حجم البيانات التي تتبعها للمستخدم.
- استخدم أدوات مراقبة الأداء عشان تعرف نقاط الضعف.

التعامل مع الـ APIs في Node.js



يعني إيه APIs؟

ال APIs هي وسيلة لتبادل البيانات بين التطبيق بتاعك وأي نظام أو خدمة ثانية في Node.js بنستخدم مكتبات زي **express** لعمل **RESTful APIs**.

خطوات إنشاء API بسيط باستخدام Express:

- تثبيت Express:

```
npm install express
```

اختبار ال API باستخدام Postman:

1. افتح Postman.
2. جرب طلبات **GET, POST**, وهكذا على الروابط اللي في الكود.
3. شوف البيانات اللي راجعة وتأكد إنها صح.

نصائح مهمة ؟



- تنظيم الكود: استخدم ال Controllers لتنظيم الكود بتاعك، زي ما عملنا في Laravel.
- استخدم أدوات زي Swagger لتوثيق ال APIs.

```
npm install swagger-ui-express
```

التأكد من صلاحيات المستخدم: استخدم **Middleware** زي **jsonwebtoken** لحماية ال APIs.

التعامل مع الأحداث (Event-Driven) (Architecture)

The background features several green, three-dimensional geometric shapes, including hexagons and chevrons, arranged in a pattern. Thin white lines connect some of these shapes, creating a network-like structure. Small white dots are scattered throughout the black background, adding to the abstract, digital aesthetic.

التعامل مع الأحداث (Event-Driven Architecture)

Node.js يعتمد على الـ Event-Driven Architecture في شغله، وده معناه إن الأحداث بتتحكم في تشغيل الكود بدل الـ sequential execution. لو عندك نظام بيحتاج يتعامل مع أحداث زي الضغط على زرار واستقبال بيانات أو انتهاء عملية الـ Event System في Node.js هيساعدك جدًا.

إزاي تستخدم الأحداث؟

- Node.js بتوفر مكتبة **built-in** اسمها **events**، واللي من خلالها تقدر تعمل الـ Event Emitter.

```
const EventEmitter = require('events');
const myEmitter = new EventEmitter();

// تعريف الحدث
myEmitter.on('sayHello', (name) => {
  console.log('Hello, ${name}!');
});

// تشغيل الحدث
myEmitter.emit('sayHello', 'Ahmed');
```

ايه الي هستفيده !



- في التعامل مع طلبات HTTP.
- تنفيذ عمليات معقدة زي قراءة الملفات أو انتظار بيانات من API.

Youssef Salah

الـ Middleware

في Express



ال Middleware في Express

Middleware في Express هو كود بيتنفذ بين استلام الطلب (Request) والرد (Response). بيكون مفيد جدًا لتنفيذ عمليات زي:

- التحقق من المستخدم (Authentication).
- تسجيل البيانات (Logging).
- التعامل مع الأخطاء.

مثال بسيط ؟

```
const express = require('express');
const app = express();

// Middleware بسيط
app.use((req, res, next) => {
  console.log('Request Type: ${req.method}');
  next(); // عشان الطلب يكمل next() لازم نستخدم
});

// Route
app.get('/', (req, res) => {
  res.send('Welcome to Middleware Example!');
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

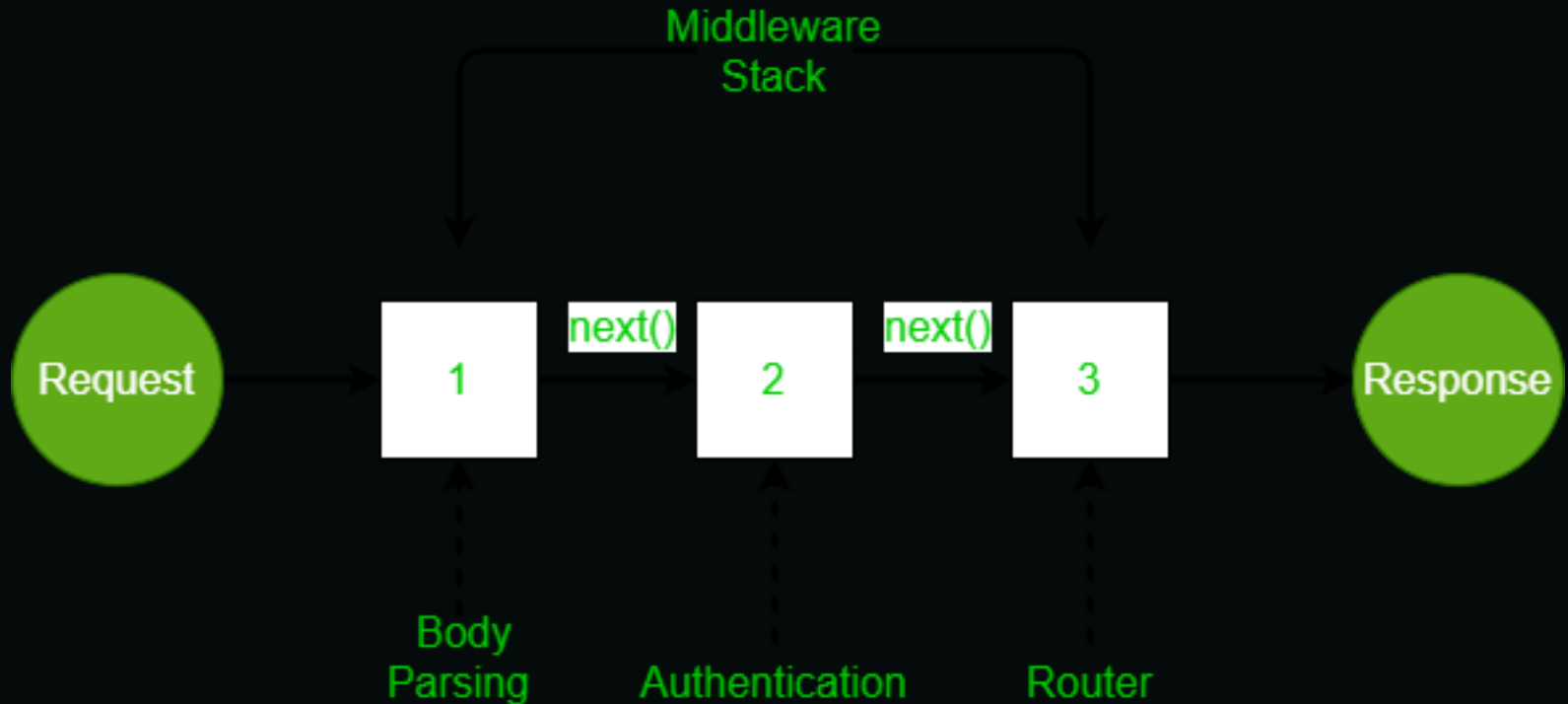
Youssef Salah

أنواع الـ Middleware:

1. Built-in Middleware: زي `express.json()` عشان تتعامل مع JSON.

2. Third-party Middleware: زي `helmet` و `cors`.

3. Custom Middleware: زي المثال اللي فوق. ▲



Youssef Salah



ال Error Handling في Node.js

الأخطاء جزء طبيعي من أي تطبيق، ومعالجة الأخطاء في Node.js ممكن تكون في:

- الكود العادي (Synchronous).
- الأكواد اللي فيها عمليات غير متزامنة (Asynchronous).

مثال Error Handling في Synchronous Code:

```
try {  
  throw new Error('Something went wrong!');  
} catch (err) {  
  console.error(err.message);  
}
```

Error Handling في Asynchronous Code باستخدام Callbacks:

```
const fs = require('fs');  
  
fs.readFile('file.txt', 'utf8', (err, data) => {  
  if (err) {  
    console.error('Error reading file:', err);  
    return;  
  }  
  console.log(data);  
});
```

Youssef Salah

Error Handling باستخدام Promises:

```
const fs = require('fs').promises;

fs.readFile('file.txt', 'utf8')
  .then((data) => console.log(data))
  .catch((err) => console.error('Error reading file:', err));
```

Error Handling باستخدام Async/Await:

```
const fs = require('fs').promises;

(async () => {
  try {
    const data = await fs.readFile('file.txt', 'utf8');
    console.log(data);
  } catch (err) {
    console.error('Error reading file:', err);
  }
})();
```

التعامل مع الـ

Streams في

Node.js



التعامل مع ال Streams في Node.js

Streams تُستخدم في Node.js عشان تتعامل مع البيانات الكبيرة بشكل أكفأ بدل ما تحمل كل البيانات مرة واحدة في الذاكرة وده مفيد جدًا مع ملفات الفيديو أو الصوت أو حتى قراءة البيانات من الشبكة.

أنواع ال Streams:

1. **Readable Streams**: عشان تقرأ البيانات.
2. **Writable Streams**: عشان تكتب البيانات.
3. **Duplex Streams**: بتقرأ وتكتب في نفس الوقت.
4. **Transform Streams**: بتعدل البيانات وهي ماشية في الستريم.

مثال على Readable Stream:

```
const fs = require('fs');

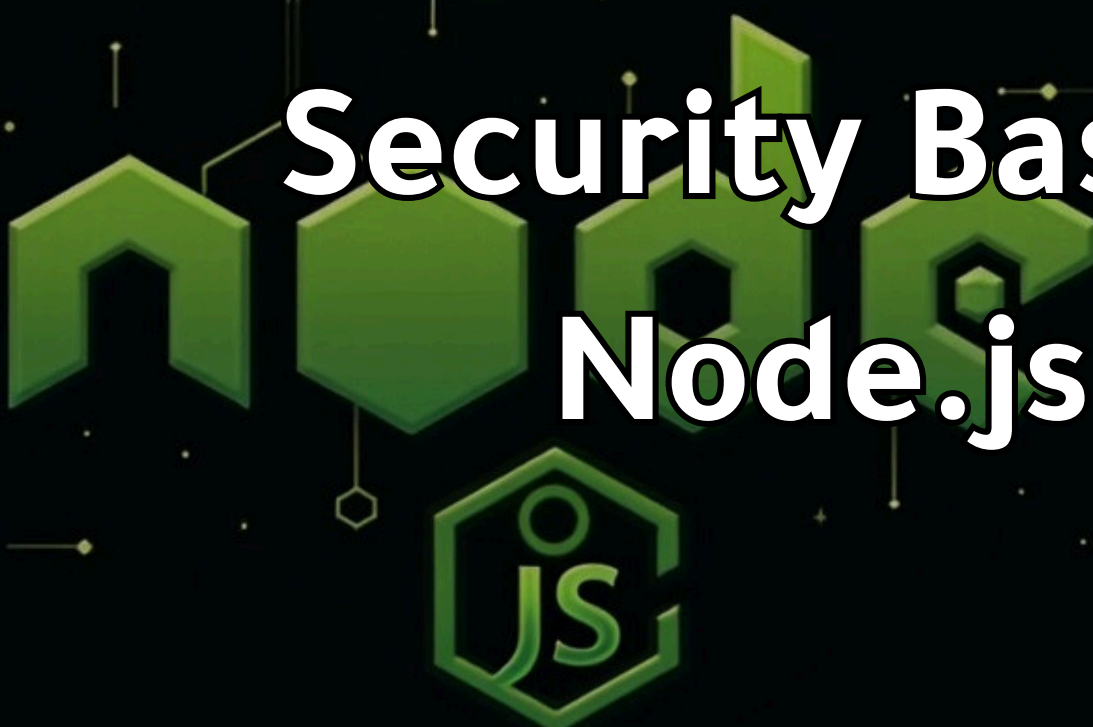
// Stream قراءة ملف باستخدام
const readableStream = fs.createReadStream('bigfile.txt', { encoding: 'utf8' });

readableStream.on('data', (chunk) => {
  console.log('Received chunk:', chunk);
});

readableStream.on('end', () => {
  console.log('Finished reading file.');
```

هتفيدك لو بتتعامل مع ملفات كبيرة جدًا وفي ال APIs اللي بتتعامل مع تحميل أو تنزيل الملفات.





Security Basics

Node.js في

Node.js في Security Basics

الأمان جزء مهم جدًا في أي تطبيق خاصة لو التطبيق يتعامل مع بيانات حساسة أو مستخدمين.

بعض النصائح الأساسية للأمان:

1. تجنب ال SQL Injection:

تأكد إن المدخلات من المستخدم بتتفلتر كويس قبل استخدامها في قاعدة البيانات.

مثال باستخدام مكتبة مثل Sequelize:

```
const User = require('./models/User');

// بدلاً من:
const query = `SELECT * FROM users WHERE username = '${username}'`;
db.query(query);

// استخدم:
User.findOne({ where: { username } });
```

استخدام ال Helmet

Helment بيضيف هيدرز أمان للطلبات.

```
const express = require('express');
const helmet = require('helmet');
const app = express();

app.use(helmet());
```

تشفير البيانات و المدخلات (Validation)

تشفير البيانات الحساسة:

استخدم مكتبات زي bcrypt عشان تشفر الباسوردز (كلمات المرون).

```
const bcrypt = require('bcrypt');

const hashPassword = async (password) => {
  const salt = await bcrypt.genSalt(10);
  return await bcrypt.hash(password, salt);
};
```

تحقق من المدخلات (Validation):

استخدم مكتبات زي Joi أو Validator.js عشان تتأكد إن المدخلات سليمة.

```
const Joi = require('joi');

const schema = Joi.object({
  username: Joi.string().min(3).required(),
  password: Joi.string().min(8).required(),
});

const result = schema.validate({ username: 'user', password: 'pass1234' });
if (result.error) {
  console.error(result.error.details);
}
```

تجنب عرض الأخطاء للمستخدمين:

لا تعرض تفاصيل الأخطاء في ال Response للمستخدم النهائي.



Youssef Salah

التعامل مع الـ WebSockets في Node.js



التعامل مع ال WebSockets

ال WebSockets يستخدم لما يكون التطبيق محتاج يتواصل في الوقت الحقيقي (Real-Time) زي الشات أو الألعاب أونلاين وال Node.js بيشتغل كويس مع ال WebSockets عشان معماريته ال Event-Driven.

إزاي نستخدم WebSockets؟

هنستخدم مكتبة مشهورة زي **socket.io**.

مثال على العميل (Client):

```
<script src="https://cdn.socket.io/4.0.0/socket.io.min.js"></script>
<script>
  const socket = io('http://localhost:3000');

  socket.on('message', (data) => {
    console.log('Server message:', data);
  });

  socket.emit('message', 'Hello, Server!');
</script>
```

ايه الي هستفيده !



• تطبيقات الشات.

• متابعة الأحداث المباشرة (Live Updates).

• الألعاب أونلاين.

Youssef Salah

التعامل مع ال Sessions و JSON Web Tokens (JWT)



التعامل مع ال Sessions و JSON Web Tokens (JWT)

Sessions تُستخدم لتخزين بيانات المستخدم على السيرفر بعد تسجيل الدخول ودي بتكون مريوطة بالمتصفح باستخدام الكوكيز.

JWT (JSON Web Tokens)

JWT يُستخدم لتأمين ال APIs. بتكون عبارة عن توكن بيتبع مع كل طلب للتحقق من المستخدم.

هنستخدم مكتبة jsonwebtoken.

```
const jwt = require('jsonwebtoken');

// إنشاء توكن
const token = jwt.sign({ id: 123, role: 'admin' }, 'mySecretKey', { expiresIn: '1h' });

// التحقق من التوكن
try {
  const decoded = jwt.verify(token, 'mySecretKey');
  console.log('Token is valid:', decoded);
} catch (err) {
  console.log('Invalid token.');
```

فين وامتني نستخدم ده؟



- ال APIs اللي محتاجة تحقق هوية المستخدم.
- التطبيقات اللي بتشتغل بـ SPA (Single Page Applications) زي React أو Angular.

Youssef Salah

أهم أسئلة ما بعد الملخص ؟

هل الملخص ده كافي لتعلم الأساسيات فقط؟

- الملخص ده بيغطي مختصر الأساسيات فقط وبيعتبر نقطة انطلاق لو حابب تتعلم أكثر يفضل تكمل كورسات أو تتعمق في المصادر الأخرى.

هل ممكن استبدال الملخص ده بكورس كامل؟

- الملخص ده مش بديل عن كورس كامل هو مجرد مقدمة سريعة ومبسطة تساعدك تبدأ بالأساسيات لكن الكورس الكامل هيووفر لك تفاصيل أكثر وتطبيقات عملية في المجال.

• هل الملخص ده هيفيدني لو كنت مبتدئ؟

- الملخص ده مثالي لو كنت مبتدئ وعايز تبدأ من نقطة أساسية فقط هيساعدك تتعرف على المفاهيم الرئيسية والأساسيات.

هل يجب عليّ تطبيق كل النقاط المذكورة في الملخص؟

- لو كنت مبتدئ، يفضل تطبق النقاط عشان يكون عندك تجربة عملية ولكن إذا كنت عايز تتعلم أكثر بعد ما تكمل الملخص ده هتحتاج تتعلم تفاصيل أعمق.

لو عجبك الملخص متنساش الدعاء ليك و لإخواتك
المسلمين خصوصا المستضعفين منهم و الاموات
منهم الي ملهمش حد يدعيلهم و متنساش طبعاً
كثرة الصلاة على النبي و جزاكم الله خيراً



اعداد: Youssef Salah